Assignment 9 — Advanced Joins Subqueries

Project: College Management System | Database: college_management_system | SQL dialect: MySQL 8.0

Based on the seven-table college design supplied for this project. Example records and expected outputs are illustrative, not screenshots from MySQL Workbench.

Multi-table reports and subqueries

The reports demonstrate joins, EXISTS, scalar subqueries and grouped summaries. Results shown are computed from the supplied fictional sample dataset.

1. INNER JOIN: student course registrations

```
SELECT s.Name AS Student, c.Course_Name AS Course, e.Semester FROM Enrollment e JOIN
Student s ON s.Student_ID=e.Student_ID JOIN Course c ON c.Course_ID=e.Course_ID ORDER BY
s.Student_ID,c.Course_ID;
```

Expected output from the included demonstration dataset (9 rows):

Student	Course	Semester
Aarav Sharma	Database Management Systems	3
Aarav Sharma	Python Programming	3
Diya Nair	Database Management Systems	3
Diya Nair	Computer Networks	3
Kabir Singh	Accounting Principles	5
Ananya Iyer	Probability and Statistics	3
Rohan Das	Database Management Systems	5
Rohan Das	Python Programming	5
Sara Khan	Accounting Principles	3

2. LEFT JOIN: departments and faculty

```
SELECT d.Department_Name, f.Name AS Faculty FROM Department d LEFT JOIN Faculty f ON
f.Department_ID=d.Department_ID ORDER BY d.Department_ID,f.Faculty_ID;
```

Expected output from the included demonstration dataset (4 rows):

Department_Name	Faculty
Computer Applications	Prof. Nisha
Computer Applications	Prof. Arjun
Commerce	Prof. Priya
Mathematics	Prof. Vivek

3. Three-table join: examination marks

```
SELECT s.Name AS Student,c.Course_Name AS Course,r.Marks,r.Grade FROM Result r JOIN Student
s ON s.Student_ID=r.Student_ID JOIN Examination x ON x.Exam_ID=r.Exam_ID JOIN Course c ON
c.Course_ID=x.Course_ID ORDER BY s.Student_ID,x.Exam_ID;
```

Expected output from the included demonstration dataset (9 rows):

Student	Course	Marks	Grade
Aarav Sharma	Database Management Systems	88	A
Aarav Sharma	Python Programming	81	A
Diya Nair	Database Management Systems	92	A+
Diya Nair	Computer Networks	79	B
Kabir Singh	Accounting Principles	84	A
Ananya Iyer	Probability and Statistics	95	A+
Rohan Das	Database Management Systems	76	B
Rohan Das	Python Programming	69	C
Sara Khan	Accounting Principles	73	B

4. Subquery: students above average mark

```
SELECT DISTINCT s.Name FROM Student s JOIN Result r ON r.Student_ID=s.Student_ID WHERE
r.Marks > (SELECT AVG(Marks) FROM Result) ORDER BY s.Name;
```

Expected output from the included demonstration dataset (4 rows):

Name
Aarav Sharma
Ananya Iyer
Diya Nair
Kabir Singh

5. Correlated subquery: students enrolled in DBMS

```
SELECT s.Student_ID,s.Name FROM Student s WHERE EXISTS (SELECT 1 FROM Enrollment e JOIN
Course c ON c.Course_ID=e.Course_ID WHERE e.Student_ID=s.Student_ID AND
c.Course_Name='Database Management Systems') ORDER BY s.Student_ID;
```

Expected output from the included demonstration dataset (3 rows):

Student_ID	Name
101	Aarav Sharma
102	Diya Nair
105	Rohan Das

6. Grouped report: course average scores

```
SELECT c.Course_Name,COUNT(r.Result_ID) AS Results,ROUND(AVG(r.Marks),2) AS Average_Marks
FROM Course c JOIN Examination x ON x.Course_ID=c.Course_ID LEFT JOIN Result r ON
r.Exam_ID=x.Exam_ID GROUP BY c.Course_ID,c.Course_Name ORDER BY c.Course_ID;
```

Expected output from the included demonstration dataset (5 rows):

Course_Name	Results	Average_Marks
Database Management Systems	3	85.33
Python Programming	2	75.0
Accounting Principles	2	78.5
Probability and Statistics	1	95.0

Course_Name	Results	Average_Marks
Computer Networks	1	79.0

7. Subquery: highest scoring student

```
SELECT s.Name,r.Marks FROM Result r JOIN Student s ON s.Student_ID=r.Student_ID WHERE
r.Marks=(SELECT MAX(Marks) FROM Result);
```

Expected output from the included demonstration dataset (1 rows):

Name	Marks
Ananya Iyer	95